

AMS Bootcamp 2026: Bash, Git, Python

Brendan McKinley

August 2026

Objectives

- 1 Familiarity with general principles of version control and Git commands
- 2 Getting comfortable with the terminal and Bash scripting
- 3 Python installed and example Jupyter notebook set up
- 4 Experience with the process of installing programs and diagnosing installation issues

Survey

- Operating systems:

Survey

- Operating systems:
 - Windows?

Survey

- Operating systems:
 - Windows?
 - Linux?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?
- How many have experience with:

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?
- How many have experience with:
 - Version control with Git?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?
- How many have experience with:
 - Version control with Git?
 - Writing and executing Bash scripts?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?
- How many have experience with:
 - Version control with Git?
 - Writing and executing Bash scripts?
 - Python scripting and Jupyter notebooks?

Survey

- Operating systems:
 - Windows?
 - Linux?
 - Mac?
- How many have already installed:
 - Git?
 - Python?
 - Visual Studio Code?
- How many have experience with:
 - Version control with Git?
 - Writing and executing Bash scripts?
 - Python scripting and Jupyter notebooks?
- How many have set up a GitHub account?

Agenda

- 1 Bash: CLI
- 2 Version Control with Git
- 3 Bash: Scripting
- 4 Python/Jupyter Notebooks

Interacting with the OS

- Shell provides a text- or UI-based interface with the underlying operating system of your machine
 - Defaults
 - Linux/macOS: BASH (Bourne-Again Shell)
 - Windows: PowerShell/Command Prompt
 - BASH can be used on Windows with Git Bash -
 - <https://git-scm.com/install>

Bash commands and keywords

- Cheat sheet – <https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet/>
- `..` – Refers to parent directory
- `ls` – List contents of current directory
- `vim` – Create/edit file
- `cd` – Change directory
- `mkdir` – Create directory
- `mv` – Move a file (or rename a file, if moving to the same directory)
- `rm` – Remove a file (or directory, when using the `-r` flag)
- `scp` – Used to securely copy files (and directories, when using the `-r` flag) between machines

Exercise: Command line interface warmup

- ❶ Open a new terminal (or Git Bash instance if on Windows).
- ❷ List the contents of the base directory.
 - `ls`
- ❸ Make a new directory.
 - `mkdir test_directory`
- ❹ Create a new text file with vim.
 - `vim test.txt`
- ❺ Put some text in the file.
 - ❶ `i`
 - ❷ `"Hello world!"`
 - ❸ `Esc`
 - ❹ `:wq`

Exercise: Command line interface warmup (continued)

- 1 List the contents of `test_directory` and confirm that `test.txt` is shown.
- 2 Move `test.txt` to the parent directory.
 - `mv test.txt ..`
- 3 Confirm that `test.txt` is present in the base directory and absent from `test_directory`.
- 4 Move `test.txt` back.
- 5 Remove both `test_directory` and `test.txt`.
 - `rm -r test_directory`

- Install guide: <https://helpcenter.mines.edu/TDClient/1946/Portal/KB/ArticleDet?ID=154280>

Integrated Development Environment (IDE)

- Centralized platform for development, debugging, testing, and version control
- Examples: Eclipse, RStudio, Matlab, Visual Studio, Visual Studio Code
- VSCode: Lightweight, versatile, good shapes and colors, remote development workflow

Exercise: Install VSCode

- ① Navigate to <https://code.visualstudio.com/> and install Visual Studio Code.
- ② Familiarize yourself with the UI.
 - Top drop-down menu
 - File → Open, File → Open Folder
 - Terminal → New Terminal
 - View → Command Palette
 - Left-hand menu
 - LHS: Explorer, Search, Source Control

Why version control?

Why version control?

- 1 Backing up your own work

Why version control?

- 1 Backing up your own work
- 2 Logically organizing your own work

Why version control?

- 1 Backing up your own work
- 2 Logically organizing your own work
- 3 Keeping track of your work's history

Why version control?

- ① Backing up your own work
- ② Logically organizing your own work
- ③ Keeping track of your work's history
- ④ Collaboration and code reviews for shared work

Why version control?

- 1 Backing up your own work
- 2 Logically organizing your own work
- 3 Keeping track of your work's history
- 4 Collaboration and code reviews for shared work
- 5 Automated testing for shared work

Basic Git workflow

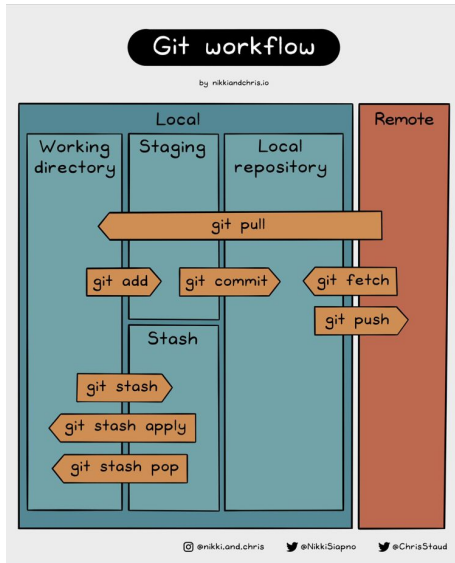


Figure: Typical Git workflow overview.

Principles of good version control

- Descriptive commit names. Like, actually
- “Commit early, commit often” but also commit consistently
- Organize by importance – tagging, feature branches, releases

Git commands

- Cheat sheet: <https://git-scm.com/cheat-sheet>
- `git init`: Initialize new repository
- `git add <file>`: Stage changes for commit
- `git commit -m "message"`: Commit staged changes with message "message"
- `git stash`: Store changes in the stash
- `git stash pop`: Restore most recently stashed changes
- `git branch`: List, create, delete branches

What will you actually use?

What will you actually use?

- `stash` commands are really useful for seeing how a change will affect different branches before you commit

What will you actually use?

- `stash` commands are really useful for seeing how a change will affect different branches before you commit
- I use the `diff` features on GitHub's web UI and VSCode constantly

What will you actually use?

- `stash` commands are really useful for seeing how a change will affect different branches before you commit
- I use the `diff` features on GitHub's web UI and VSCode constantly
- GitHub's web UI allows you to see the history of commits affecting a specific file or folder

Exercise: Install Git

- 1 Navigate to `https://git-scm.com/install` and install Git for your OS.
- 2 Once Git is installed, run `git --version` in your terminal to verify the installation.
- 3 If you don't already have one, create a new GitHub account.
- 4 Configure your local Git to use this account.
 - `git config --global user.name "Your Name"`
 - `git config --global user.email "your.email@example.com"`

Exercise: Clone a repository

- 1 Navigate to
`https://github.com/brendan-e-mckinley/bootCampAMS/`
- 2 Fork the repository – create a copy on your GitHub account.
- 3 Clone via the terminal with `git clone`
`https://github.com/<username>/bootCampAMS.git`
 - On the GitHub web UI, the “Code” dropdown gives you several options for cloning a repository.
 - You can open a terminal directly from VSCode.
- 4 Open the repository in VSCode.

Exercise: Make a new branch

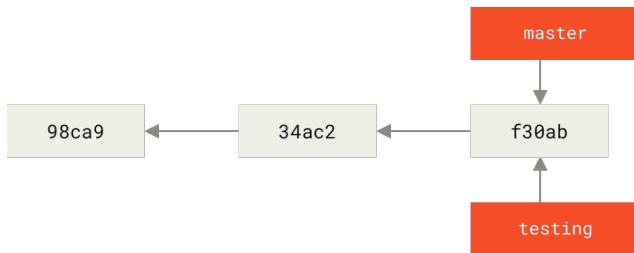


Figure: The result of running `git branch testing`. Credit: <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>

- 1 In your terminal, run `git checkout -b <firstname>_<lastname>`
- 2 Verify that the message "Switched to a new branch '`<firstname>_<lastname>`'" is returned
- 3 Push your new branch using the VSCode UI
 - Or via the terminal: `git push --set-upstream origin <firstname>_<lastname>`

Exercise: .gitignore

- 1 Update the .gitignore file so that the repository excludes any files with the .png extension.
- 2 Add a .png file to the repository.
- 3 Ensure the image is excluded from the “Source Control” tab in VSCode.
- 4 Commit and push your update to the .gitignore file to your branch.

Exercise: Open a pull request

- 1 Navigate to `https://github.com/brendan-e-mckinley/bootCampAMS/`
- 2 Select the “Pull requests” tab.
- 3 Select “New pull request”.
- 4 Select “Compare across forks”.
- 5 Change the “Head repository” branch to your working branch.
- 6 Create the pull request.

Exercise: Create a Repository

- 1 Go to your account page on GitHub.
- 2 Select the “Repositories” tab.
- 3 Select “New”.
- 4 Name the repository and give it a description.
- 5 Notice the visibility, README, and `.gitignore` settings.

Break

Python scripting

- Structure

- Whitespace matters!
- Main entry point

```
if __name__ == "__main__":
```

- Function definition

```
def greater_than(arg_1, arg_2):
```

- Variable assignments

```
arg_1 = 10  
arg_2 = 1
```

Exercise: Running a Python script

- ❶ If you haven't already, install Python.
 - ❶ Download the relevant Python installer from <https://www.python.org/downloads/>
 - ❷ Add Python to PATH if you're installing on a Windows machine.
 - ❸ Verify the Python installation with `python3 --version` or `python --version`
- ❷ Run a script from the terminal.
 - ❶ Navigate to the `src` directory of the `bootCampAMS` repository.
 - ❷ Execute the `greater.py` Python script using the default Python installation with `python greater.py`

Exercise: Running a Python script (continued)

- ❶ Install Python extensions from the “Extensions” tab on the left-hand menu.
- ❷ Run a script from VSCode.
 - ❶ Open the `greater.py` Python script in VSCode.
 - ❷ Open the Command Palette with “View” → “Command Palette” or `Cmd + Shift + P`
 - ❸ Click “Python: Select Interpreter” and select the default interpreter for VSCode.
 - ❹ Click “Run Python File” in the upper-right corner of the VSCode UI.

Exercise: Debugging a Python script

- 1 Open the `greater.py` Python script in VSCode.
- 2 Place a breakpoint on the `greater_than()` function call.
- 3 Select "Python Debugger: Debug Python File" from the "▶" dropdown in the upper-right corner.
- 4 Step into the `greater_than()` function.
- 5 Hover over the `arg_2` variable to see its contents.
 - Notice the "Run and Debug" information on the left-hand side.

Bash scripting

- You can automate any command line Bash commands by organizing them in a Bash script.
- Useful for:

Bash scripting

- You can automate any command line Bash commands by organizing them in a Bash script.
- Useful for:
 - Repeated operations

Bash scripting

- You can automate any command line Bash commands by organizing them in a Bash script.
- Useful for:
 - Repeated operations
 - Logically connecting and sharing information between different Bash commands

Bash scripting

- You can automate any command line Bash commands by organizing them in a Bash script.
- Useful for:
 - Repeated operations
 - Logically connecting and sharing information between different Bash commands
 - Communicating with third party applications like a GitHub CI/CD pipeline

Exercise: Running a bash script

- 1 Open the `execute_operation.sh` script in the `src` directory of the `bootCampAMS` repository in Visual Studio Code.
 - Notice the shebang followed by the Bash path: `#!/bin/bash`
- 2 Navigate to the `src` directory in the terminal.
- 3 Try to execute the script with `./execute_operation.sh`
- 4 Make the script executable with `chmod +x execute_operation.sh`
 - More information on `chmod`:
<https://www.redhat.com/en/blog/introduction-chmod>
- 5 Execute the script with `./execute_operation.sh`

Aside: What's wrong with `operation.py`?

- Learn more: <https://peps.python.org/pep-0008/>

Virtual environments

- Package: a folder with some Python code, along with everything you need to use that code in your projects
- Certain packages will be useful for many different Python projects
 - matplotlib
 - numpy
- Others will only be relevant for specific projects
- Virtual environments allow you to customize package installations for specific projects
- Learn more: <https://docs.python.org/3/library/venv.html>

Exercise: Setting up a virtual environment

- 1 In your terminal, navigate to the base directory of the bootCampAMS project.
- 2 Create a new virtual environment with `python -m venv test-env`.
- 3 Activate the environment:
 - BASH (Linux/MacOS): `source test-env/bin/activate`
 - Command Prompt (Windows): `test-env\Scripts\activate.bat`
- 4 Ensure that the terminal now lists `(test-env)` before the current active directory.
- 5 Install `pytest` in your virtual environment with `pip install pytest`.

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic
- Testable code

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic
- Testable code
 - Atomic functions

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic
- Testable code
 - Atomic functions
 - Single-responsibility classes and objects

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic
- Testable code
 - Atomic functions
 - Single-responsibility classes and objects
 - Verifiable success and failure

Testing in Python

- Why test, especially if you're not writing production code or contributing to a shared codebase?
 - Prove to yourself and others that your code does what you think it does
 - The “bug space” expands quickly as you build out a codebase – narrow down culprits
 - Writing “testable code” can be a good organizational tactic
- Testable code
 - Atomic functions
 - Single-responsibility classes and objects
 - Verifiable success and failure
 - SOLID design principles –
<https://realpython.com/solid-principles-python/>

Exercise: Python testing

- 1 Open `test/test_greater.py`
- 2 Open a terminal in VSCode and navigate to the top-level directory of the repository (`/bootcampAMS`).
- 3 Ensure that your `(test-env)` virtual environment is activated.
- 4 Run `python -m pytest`.

Exercise: Environment from file

- 1 Conda is a third-party package and environment manager.
- 2 Install Conda – `https://www.anaconda.com/docs/getting-started/installation`
 - Or, if you hate that, you're valid
 - Alternative: Micromamba – `https://mamba.readthedocs.io/en/latest/installation/micromamba-installation.html`
 - Alternative: `venv` – Should have everything you need to create a virtual environment from which you can run the notebook
 - Required dependencies are listed in `environment.yml`

Exercise: Environment from file (continued)

- 1 Once Conda is installed, you can build a new virtual environment from the `environment.yml` file with:
 - Conda – `conda env create -f environment.yml`
 - Micromamba – `micromamba create -f environment.yml`
- 2 Deactivate the `test-env` virtual environment by running `deactivate`
- 3 Activate the `jupyter-env` environment you just created:
 - Conda – `conda activate jupyter-env`
 - Micromamba – `micromamba activate jupyter-env`

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:
 - Caching and memory

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:
 - Caching and memory
 - Mathematical notation and Latex

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:
 - Caching and memory
 - Mathematical notation and Latex
 - Helping others get a deeper understanding of shared code

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:
 - Caching and memory
 - Mathematical notation and Latex
 - Helping others get a deeper understanding of shared code
 - Presenting findings

Jupyter notebooks

- Combine Markdown, Python code, and plots in a single readable notebook.
- Useful for:
 - Caching and memory
 - Mathematical notation and Latex
 - Helping others get a deeper understanding of shared code
 - Presenting findings
 - Organizing code for your own benefit

Exercise: Run a Jupyter notebook

- 1 Install the Jupyter extension from the “Extensions” tab on the left-hand menu of VSCode.
- 2 In the top-right corner of `notebook.ipynb`, choose the `jupyter-env` Python kernel.
- 3 Run each section of the Jupyter notebook in order. Notice:
 - Run/debug cell
 - Edit cell – markdown cells interpret Latex code directly
 - Split cell